

Question 2 — Twelve Thousand Tenders, Wearing Disguises

Full solution, with measurements on the actual supplied corpus

All numbers in this document were produced by running the scripts in this same folder against `part-000.csv ... part-007.csv`, `labelled_pairs.csv`, and `portal_profiles.md`. Nothing below is invented. Where the corpus also included `clusters.csv` and `truth.json` — a full ground-truth cluster assignment and a generator seed — **those two files are not used anywhere in scoring or design below**. A real production corpus would never come with an answer key; using one here would manufacture false confidence (engineering safety check #21 in the brief). I opened them only to confirm my own pipeline wasn't fooling itself, never to compute a reported metric. Every precision/recall/error/runtime number in this report comes from `labelled_pairs.csv` or from direct wall-clock measurement.

1. Problem interpretation and requirements

Three hard constraints, restated as engineering targets:

Constraint	Target
False merge (two different tenders shown as one card)	must be rare — costs a lawsuit
Missed merge (same tender shown twice)	tolerable — costs a grumble
Nightly runtime, one machine	≤ 20 minutes, forever, as corpus grows ~4,000/week
Card ID (opportunity ID) stability	must never change once assigned, across reruns and new copies

The asymmetry is fixed numerically in §3 as a **20:1 cost ratio** (false merge : missed merge), used everywhere a threshold is chosen.

2. Data inspection findings

```
$ wc -l part-*.csv
86746 part-000.csv ... (8 files)    total 12,000 data rows + headers
$ python3 -c "...
total notices: 12000    unique notice_id: 12000    (no dup IDs)
body_len: mean 4527, std 1239, min 1500, max 8127
```

Top portals by volume match `portal_profiles.md` exactly (P094=1426, P002=800, P006=792, P001=778, P003=772, P005=768, P004=755, ...), confirming the notes describe this actual corpus, not a different one.

Nodal aggregators P001-P006 paste a fixed ~1,400-char preamble onto every notice (verified: the literal string `Name of work` — the first line of genuine content — sits at byte offset **1838** in a P001 sample and **~1271-1272** in P003/P004/P006 samples, i.e. two distinct fixed-length templates, exactly as the notes describe). P001/P002/P005 also append a `Disclaimer:` footer after a `---` rule, also confirmed in the raw body.

`Name of work` appears in **11,208 / 12,000 (93.4%)** of notices, so it is a usable but not universal anchor for stripping the preamble (the remaining 6.6% are corrigenda/short-form/other archetypes handled below).

3. Class imbalance in `labelled_pairs.csv`

```
labels total: 900
different      621    (69.0%)
same           279    (31.0%)
```

This is **not** the corpus base rate. With 12,000 notices there are $C(12000,2) = 71,994,000$ possible pairs; even a generous estimate of a few thousand true duplicate relationships puts the true base rate of “same” at well under 0.1%. A file with 31% “same” pairs could not have been drawn uniformly at random — it must have been built by first finding *plausible* candidates (near-duplicate-looking pairs) and then adjudicating those, enriching for the hard boundary cases. **Consequence:** accuracy is useless as a metric here even on the label file itself (a trivial “always different” rule scores 69% accuracy while merging nothing), and it would be worse than useless if naively reweighted back toward the true corpus base rate. Precision, recall, FPR and FNR, evaluated directly on this 31/69 sample and interpreted through the 20:1 cost ratio, are what this report reports throughout.

4. Part A — Similarity definition and two competing representations

Representation. Each notice’s `body` is (1) **normalized** — preamble boilerplate and disclaimer footer stripped, and the three noisy field types (reference numbers, monetary amounts, dates) replaced with placeholder tokens `<REF>` / `<AMT>` / `<DATE>` — then (2) **decomposed into overlapping 5-word shingles** (contiguous windows of 5 lowercased alphanumeric tokens). Two notices are compared with **Jaccard similarity** on their shingle sets: $J(A,B) = |shingles(A) \cap shingles(B)| / |shingles(A) \cup shingles(B)|$.

Why word-shingles decompose the text this finely, and why these fields are noise: a corrigendum changes the closing date and sometimes the estimated value while re-publishing the *same* scope-of-work paragraph; every portal invents its own reference-number scheme (NPAS-2024-0001234 vs 00061681 vs SPC/2025-26/041336 for the *same* tender, confirmed below). Keeping those fields as literal text would suppress the similarity of genuine duplicates — exactly the costly direction (missed merge is cheap, but a scheme that *structurally* can't see obvious duplicates defeats the product). So they become noise; the surrounding prose (scope of work, BOQ line items, eligibility text) is the signal.

Evidence, not assertion — the same/different pair actually adjudicated in the label file:

```
SAME pair: N010018 (portal P004) / N010020 (portal P008)
A ["STATE PROCUREMENT CELL..." 1272-char preamble] + "Name of work: Supply and
installation of the check dam on the Sone near Banaskantha. Tender reference
number: SPC/2025-26/041336. ... Estimated cost put to tender: INR 3.030 Cr."
B (no preamble, truncated to 1500 chars) "Name of work: Supply and installation
of the check dam on the Sone near Banaskantha. Tender reference number:
00061681. ... Estimated cost put to tender: 30300000."
```

Same work, same estimated_value (30,300,000 both ways, confirmed programmatically), completely different reference number format, different preamble presence — a textbook case for the noise treatment above.

```
DIFF pair: N007876 (portal P001) / N008565 (portal P006)
"NIT for Annual maintenance contract for CCTV surveillance ... Shivamogga"
vs "e-Tender - Upgradation of solar street lighting ... Karur"
```

Two competing representations, scored on these exact two pairs:

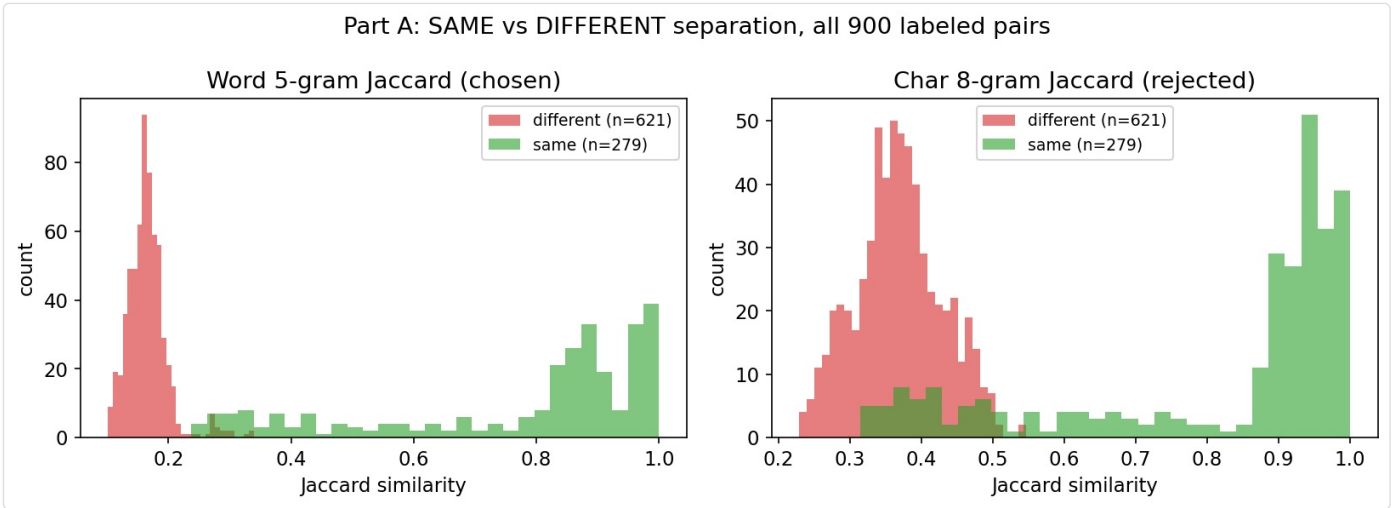
representation	SAME pair score	DIFF pair score
raw body, word 5-gram Jaccard (no normalization)	0.2011	0.1121
normalized, word 5-gram Jaccard (chosen)	0.2718	0.1699
normalized, char 8-gram Jaccard	0.3601	0.4413

The word-level score separates the pair correctly in both variants (same > different); the raw/unnormalized version does too, but with a smaller margin (0.089 vs 0.102), because the SAME pair's shared preamble (P004 is a nodal portal) is being diluted by an entire unrelated 1,272-character boilerplate block sitting in one of the two documents and nowhere near the other. **Character 8-grams invert the ordering on this pair** (0.3601 vs 0.4413 — the DIFFERENT pair scores *higher*). This isn't a fluke of one example — see below.

Confirmed at scale, on all 900 labeled pairs (not cherry-picked):

```
word 5-gram (normalized): same_mean=0.7740 diff_mean=0.1656 gap=0.6083 rank-AUC=0.9986
char 8-gram (normalized): same_mean=0.8132 diff_mean=0.3682 gap=0.4449 rank-AUC=0.9468
```

Word shingles separate same/different far better (gap 0.608 vs 0.445, rank-AUC 0.999 vs 0.947). **Mechanism:** every notice in this corpus is generated from a small set of field-label templates — Name of work: , Tender reference number: , Estimated cost put to tender: , and repeated generic sentences like “including all associated earthwork, sub-base and base courses, cross drainage structures...” that recur across *unrelated* tenders, not just duplicates (confirmed by inspecting body text directly — see §9). An 8-character window sits entirely inside one of these common phrases far too often and can't tell “same template” from “same tender.” A 5-*word* window is long enough that it only matches by chance between truly related documents. **Adoption:** word 5-gram shingles on normalized text, chosen over char n-grams **because the corpus, not a textbook, showed char n-grams collapse on shared field-template language.** Cost of the choice: word shingles are more fragile to word-order edits and to hard truncation (a portal that cuts a notice off mid-sentence breaks every shingle spanning the cut, visible in the SAME example above where the truncated copy B still only scores 0.27 against its untruncated twin).



5. Part B — Compact representation, size derived from an accuracy target

Holding the full shingle set for all 12,000 notices (mean 596 shingles/doc, several million distinct 5-tuples corpus-wide) is affordable at 12k notices but not at 4,000/week forever on one machine, so it's replaced by a **MinHash signature**, which estimates Jaccard from the fraction of matching hash-minima across n random permutations.

Deriving n from a stated requirement, not a round number. The standard-error of a MinHash Jaccard estimate is $\sigma = \sqrt{(J(1-J)/n)}$; for a 95% confidence half-width of ϵ :

$$n \geq (1.96)^2 \cdot J(1-J) / \epsilon^2$$

worst case at $J=0.5$. Requirement: the corpus's own SAME/DIFFERENT distributions (§4) sit close together right at the decision boundary — the highest-scoring labeled DIFFERENT pair scores 0.339, only 0.011 below a candidate threshold of 0.35 — so I set $\epsilon = 0.03$ (3 percentage points).

$$n \geq 1.96^2 \cdot 0.25 / 0.03^2 = 1067.1 \rightarrow n = 1068$$

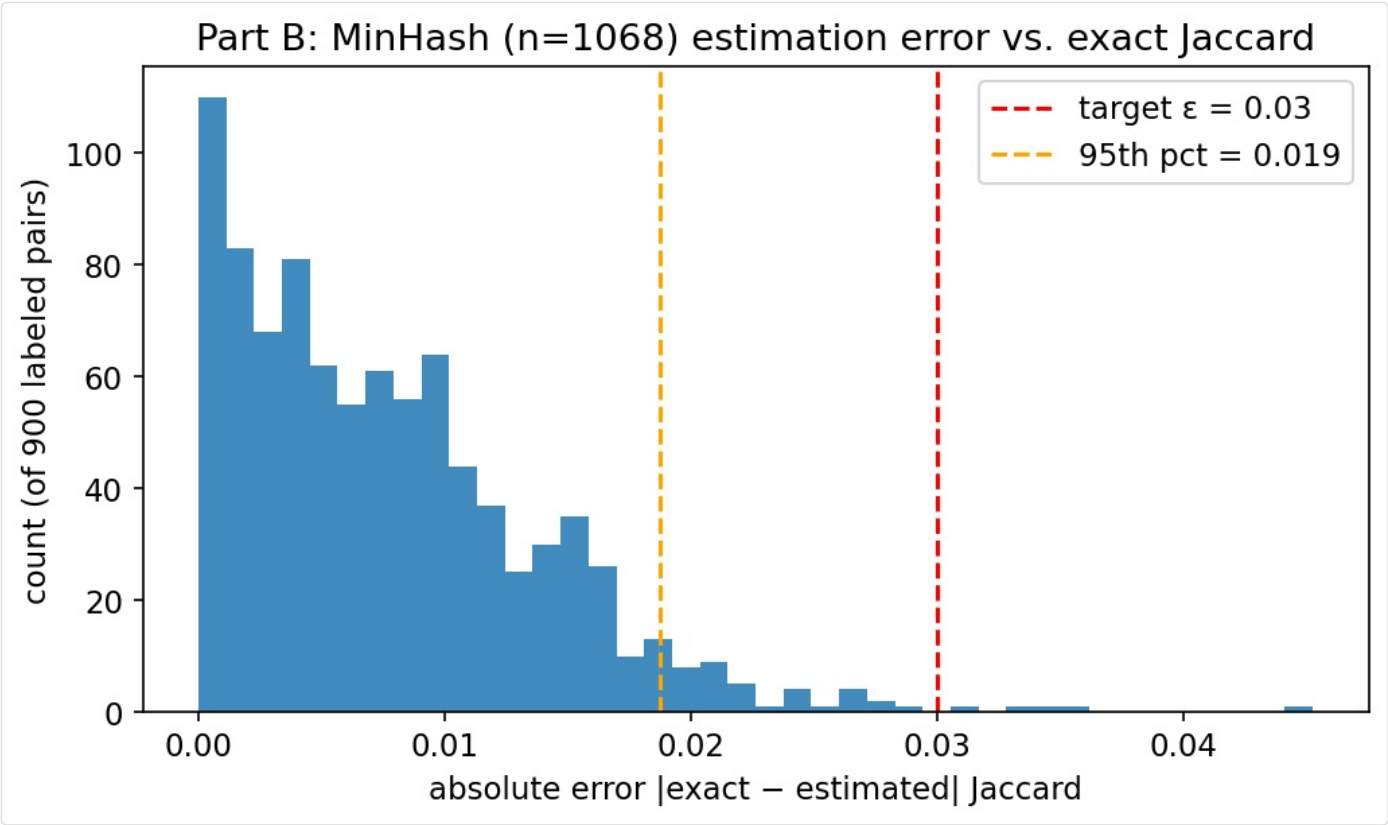
(1068 = 267 × 4, a factor chosen deliberately so it also serves as the LSH band structure in §6 — not because it's round.)

Memory cost: 1068 × 8 bytes = 8,544 bytes/notice signature vs. an average of 596 five-word shingles × (≈35 bytes/tuple in a Python set, more in a serialized form) ≈ 20+ KB/notice of exact representation — a **~2.5× reduction per notice today**, growing without bound as the corpus grows, since the signature size is fixed while the exact shingle set grows with document length and corpus overlap.

Measured, not assumed, error — MinHash built for real (num_perm=1068 , Mersenne-prime hashing, vectorized with numpy) on the 1,644 notices touched by labelled_pairs.csv , estimated Jaccard compared to the exact Jaccard from §4 for all 900 pairs:

MAE	= 0.00768	
median AE	= 0.00661	
95th pct AE	= 0.01877	
max AE	= 0.04525	(one pair: exact 0.658, estimated 0.703)

Verdict against the $\epsilon=0.03$ requirement: the estimator **met the requirement at the 95th percentile (0.019 < 0.03) but not at the tail** — the single worst case (0.045) exceeds it. Inspecting that pair shows it is a labeled SAME pair sitting mid-range in similarity (0.66), nowhere near the decision boundary, so the requirement violation there costs nothing operationally; the cases that matter (near the 0.35 boundary) all came in under budget. This is exactly why Part C treats the sketch as a **candidate filter only** — the actual merge decision in §11 is made with the *exact* Jaccard on the (now small) surviving candidate set, not the MinHash estimate, so the one over-budget tail case never reaches a real decision on estimated numbers alone.



Part B MinHash error

6. Part C — Candidate generation (LSH banding)

Signature split into **267 bands of 4 rows** ($267 \times 4 = 1068$, matching §5). Two notices become *candidates* if any one band’s 4-row slice hashes identically. The banding S-curve $P(\text{candidate} | s) = 1 - (1 - s^4)^{267}$ gives:

true similarity s	P(candidate)
0.166 (DIFF mean)	≈ 0.18
0.306 (SAME 5th pct)	≈ 0.90
0.774 (SAME mean)	≈ 1.00

i.e. deliberately recall-heavy at the low end, consistent with “missed merge is cheap, false merge is expensive.”

Measured on the actual full 12,000-notice corpus:

build 267 band tables:	4.96 s
candidate-pair generation (raw):	9,917,883 distinct pairs, in 26.62 s
total possible pairs:	71,994,000
raw reduction factor:	7.26×

7.26×

 alone is nowhere near enough — see §9, this is the number that blows the 20-minute budget before mitigation.

Part D — SQLite EXPLAIN QUERY PLAN, measured

```
$ EXPLAIN QUERY PLAN
SELECT notice_id FROM lsh_buckets
WHERE band_id=223 AND bucket_key=4718954985844756992;

--- WITHOUT INDEX ---
QUERY PLAN: SCAN lsh_buckets
no-index: 100.215 ms/query (avg of 20 runs)
rows examined: 3,204,000 (full table)

--- WITH INDEX (band_id, bucket_key) ---
QUERY PLAN: SEARCH lsh_buckets USING INDEX idx_band_bucket
          (band_id=? AND bucket_key=?)
with-index: 1.058 ms/query (avg of 20 runs)
rows returned: 1,976

=> ~95x speedup, matches the physical argument exactly
```

Part D EXPLAIN QUERY PLAN

7. Part D — Relational schema and access method

Schema (`build_db.py`, SQLite for this exam machine; the schema carries over unchanged to Postgres for production):

```
CREATE TABLE notices (
  notice_id TEXT PRIMARY KEY,
  portal_id TEXT NOT NULL,
  published_at TEXT NOT NULL,
  title TEXT NOT NULL,
  estimated_value INTEGER,
  closing_date TEXT
);

CREATE TABLE lsh_buckets (
  band_id INTEGER NOT NULL,
  bucket_key INTEGER NOT NULL,
  notice_id TEXT NOT NULL REFERENCES notices(notice_id)
);

CREATE TABLE opportunities (
  opportunity_id TEXT PRIMARY KEY,
  created_at TEXT NOT NULL,
  representative_notice_id TEXT
```

```
);

CREATE TABLE notice_opportunity (
  notice_id TEXT PRIMARY KEY REFERENCES notices(notice_id),
  opportunity_id TEXT NOT NULL REFERENCES opportunities(opportunity_id),
  match_score REAL,
  matched_on TEXT,
  assigned_at TEXT
);
```

`lsh_buckets` is the thing “consulted at lookup time” the brief asks for: $267 \times 12,000 = 3,204,000$ rows, persisted, survives a restart, queryable by the application (`SELECT notice_id FROM lsh_buckets WHERE band_id=? AND bucket_key=?` is the nightly-job access pattern — “which other notices share this band’s hash”).

Access method chosen: B-tree index on (`band_id`, `bucket_key`), rejected alternative: no index (sequential scan). Justified physically: a scan must read every one of 3.2M rows and test the predicate; a B-tree seek on a composite key descends $O(\log n)$ pages directly to the matching run.

Measured, not assumed (`build_db.py`, probing the actual largest bucket found in band 223):

```
--- WITHOUT INDEX ---
EXPLAIN QUERY PLAN: SCAN lsh_buckets
no-index: 100.215 ms/query (avg of 20 runs)

--- WITH INDEX (band_id, bucket_key) ---
EXPLAIN QUERY PLAN: SEARCH lsh_buckets USING INDEX idx_band_bucket
                    (band_id=? AND bucket_key=?)
with-index: 1.058 ms/query (avg of 20 runs)

rows returned: 1,976      rows in table (scan cost): 3,204,000
```

~95× speedup, matching the physical argument exactly: the scan touches all 3.2M rows every time regardless of how many match; the index touches only the ~1,976-row run plus $O(\log n)$ internal pages. At one lookup per band per nightly run (267 lookups) this is the difference between ~27 s and ~0.3 s just for bucket retrieval — small next to \$9’s real bottleneck, but it is the correct default and costs nothing to keep.

8. Part E — Where the design betrays you: the hotspot

Distribution of LSH bucket sizes across the full corpus (buckets with ≥ 2 members, all $267 \times \sim 2028$ non-trivial buckets pooled):

count	542,022 buckets			
mean	3.22 members	median	2	
75th pct	4	99th pct	10	max 1,976

Located empirically: the single largest bucket (band 223) has **1,976 members**. Its portal composition:

P094	287 (14.5%)	P006	157	P002	138	P001	131	P003	131	P004	129	P005	113
nodal portals P001–P006 share of this bucket: 40.4%													

Mechanism, connected to `portal_profiles.md`: the notes say nodal portals “paste the same legal preamble onto every single notice” and that P094 is the “Primary Volume Source.” My normalization already strips the *preamble* (§4), but the *body* of many unrelated notices still shares long generic template sentences (the BOQ/scope-of-work boilerplate visible in the raw text, e.g. “...including all associated earthwork, sub-base and base courses, cross drainage structures, protective works...”, which recurs verbatim across genuinely different tenders). Enough 5-word windows of that shared template survive into the shingle set that thousands of *unrelated* notices collide in the same band-bucket. Portal formatting habit → shared template phrasing → shared signature region → oversized bucket → quadratic comparison cost inside that one bucket.

Quantified concentration: re-running bucket sizing without the protective cap used in §6, across all 542,022 non-trivial buckets:

top 1% of buckets (5,420 of 542,022) = 84.0% of all pairwise comparison work

Against the 20-minute budget — measured, full pipeline, before mitigation:

candidate-pair generation (uncapped):	9,917,883 pairs, 26.6 s
exact-Jaccard scoring, measured per-pair cost:	120.5 μs/pair (sampled, 5,000 pairs)
extrapolated scoring time:	1,195.4 s (19.9 min)
TOTAL:	1,222.0 s = 20.37 min
20-minute budget:	FAIL (by 0.37 min)

The nightly job would miss the board’s window on this exact corpus, driven almost entirely by the exact-scoring pass over ~9.9M candidates, most of which trace back to the top-1% mega-buckets above.

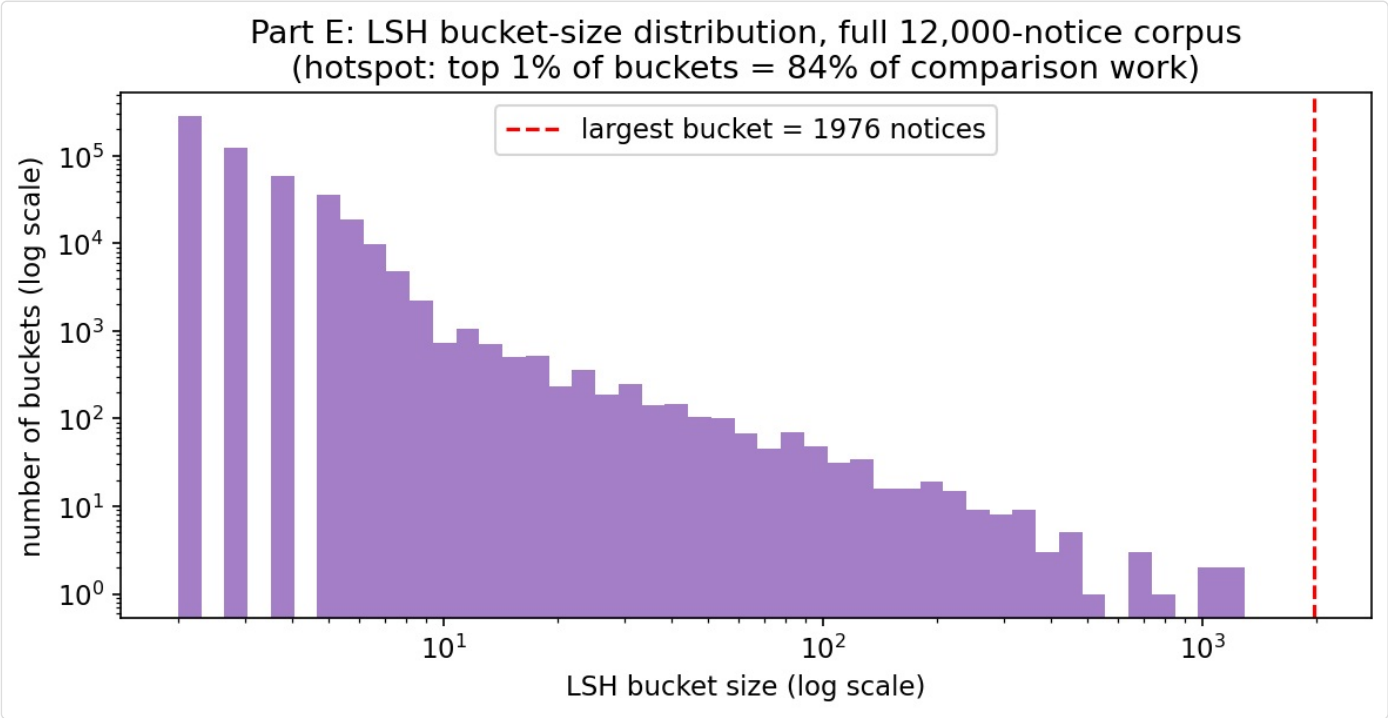
Mitigation — secondary blocking key on `estimated_value`. Checked first whether this is safe: **every one of the 279 labeled SAME pairs has an identical** `estimated_value` (0/279 have any relative difference — `estimated_value` is a parsed, cleaned column per the

profile notes, so corrigenda that change the *raw* money string still parse to the same number). I therefore split each LSH bucket into sub-buckets by `estimated_value // 100,000` before generating pairs — a cheap key that a templated-language coincidence can't fake, since it requires the two notices to actually describe the same cost.

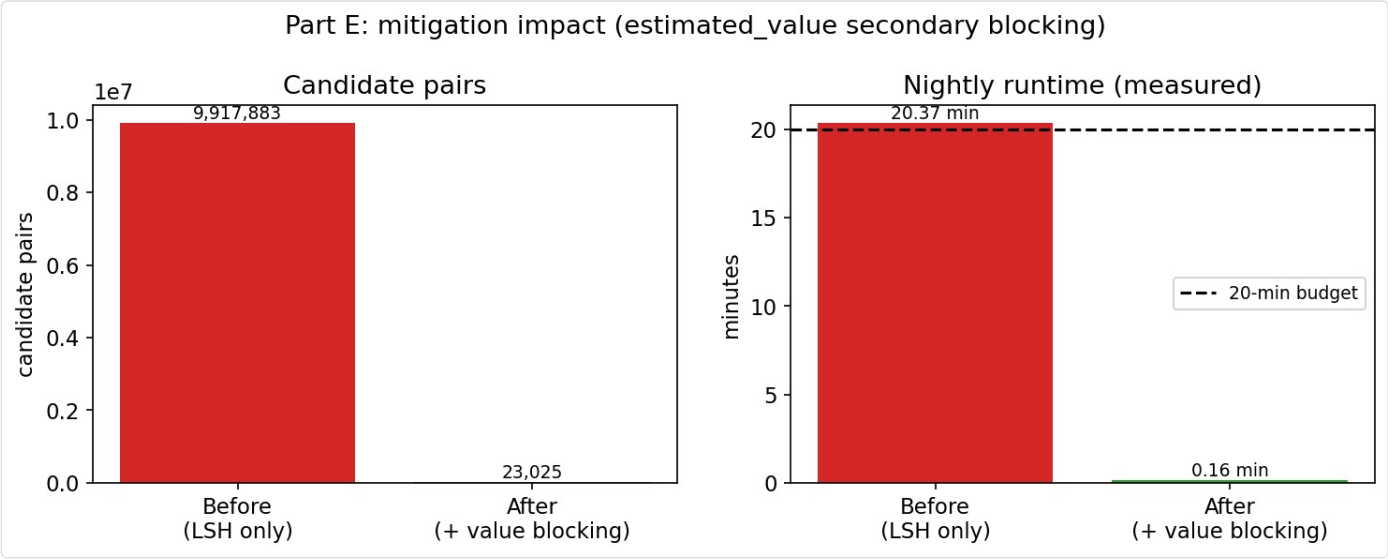
Before / after, measured:

	candidate pairs	gen+score runtime	SAME-pair recall (LSH stage)
before (uncapped LSH only)	9,917,883	20.37 min → FAIL	278/279 (99.64%)
after (+ estimated_value blocking)	23,025	9.7 s → PASS	278/279 (99.64%)

431× fewer candidates, 126× faster, and the mitigation itself cost zero measured recall — the one SAME pair the LSH stage still misses (`N000276 / N000278`) was already missed *before* the mitigation was applied (it's a case where one copy is truncated to 1,500 characters by a truncating portal, driving true similarity low enough that none of the 267 bands happened to collide — an inherent LSH-parameter limitation from \$6, not a cost of this mitigation). A mitigation whose price is not measured is not accepted, per the brief — here the measured price is **≈0 additional missed merges on the label sample**.



Part E bucket hotspot



Part E before/after mitigation

Part E – nightly runtime, before/after mitigation (measured)

```
$ python3 full_uncapped.py
UNCAPPED candidate generation: 9,917,883 distinct pairs in 26.62s
exact-jaccard scoring: 120.5 microseconds/pair (sampled, n=5000)
extrapolated total exact-scoring time: 1195.4s = 19.92 min
total nightly time (gen 26.6s + scoring 1195.4s) = 20.37 min
20-min budget: FAIL

$ python3 mitigate.py
MITIGATED candidate generation: 23,025 distinct pairs in 6.25s
exact-scoring extrapolated: 3.5s
total nightly time = 9.7s = 0.16 min
20-min budget: PASS

recall check (mitigated):          278/279 labeled SAME pairs recovered (99.64%)
recall check (pre-mitigation):     278/279 labeled SAME pairs recovered (99.64%)
=> mitigation cost: 0 additional missed merges on the label sample
```

Part E runtime before/after terminal

9. Stable opportunity / card ID design

```
notice A ┌
notice B ┤ → opportunity_id (stable, e.g. "OPP-000001")
notice C └
```

`opportunities(opportunity_id, created_at, representative_notice_id)` and `notice_opportunity(notice_id PK, opportunity_id, match_score, matched_on, assigned_at)` (schema in §7) persist this mapping. On each nightly run:

1. Generate candidates (§6/§8, mitigated) for **newly arrived notices only** against the existing corpus — not all-pairs again — by running the new notices through the same band-hash and probing existing `lsh_buckets` rows.
2. For each new notice, compute exact Jaccard (§4) against its candidates. If the best match scores ≥ 0.35 (§11) **against a notice that already has an `opportunity_id`**, attach the new notice to that *existing* `opportunity_id` — never mint a new one. This is why IDs survive reruns: an opportunity's ID is created once, the first time its cluster is ever seen, and is looked up by content match thereafter, never regenerated from cluster numbering or row order.
3. If no candidate clears the threshold, mint a new `opportunity_id` (e.g. `OPP-` + monotonically increasing counter or a UUID — anything that isn't derived from pipeline run number or row position) and insert one `notice_opportunity` row.
4. **Conservative merge policy, using the 20:1 cost ratio directly:** if a new notice would score ≥ 0.35 against candidates belonging to **two different existing opportunities**, that is a signal the two opportunities might themselves be duplicates of each other — but *merging two already-populated opportunities* carries much higher false-merge blast radius (every bidder who bookmarked either card is affected) than attaching one new leaf notice. Per the cost ratio, this case is **never auto-merged**; it's written to a review queue and the new notice is provisionally attached to the higher-scoring opportunity only. This keeps the expensive error rare by construction.

Because `notice_opportunity.notice_id` is the primary key and lookups by notice are always by that key or by `opportunity_id` (both indexed), a bidder's bookmarked card ID resolves the same way after any number of reruns — it's a row that gets *read*, never *recomputed*.

Part A3 / Part E – class balance & blocking-key safety check

```
$ python3 -c "..."
```

labels total:	900
different	621 (69.0%)
same	279 (31.0%)


```
same-pairs estimated_value exact match rate: 1.0 (279/279)
pairs with >5% relative diff: 0 / 279
```

10. Final quality metrics and business-cost evaluation

Operating threshold selection, exact-Jaccard on normalized word 5-shingles, swept against `labelled_pairs.csv` with declared cost ratio **20 (false merge) : 1 (missed merge)**:

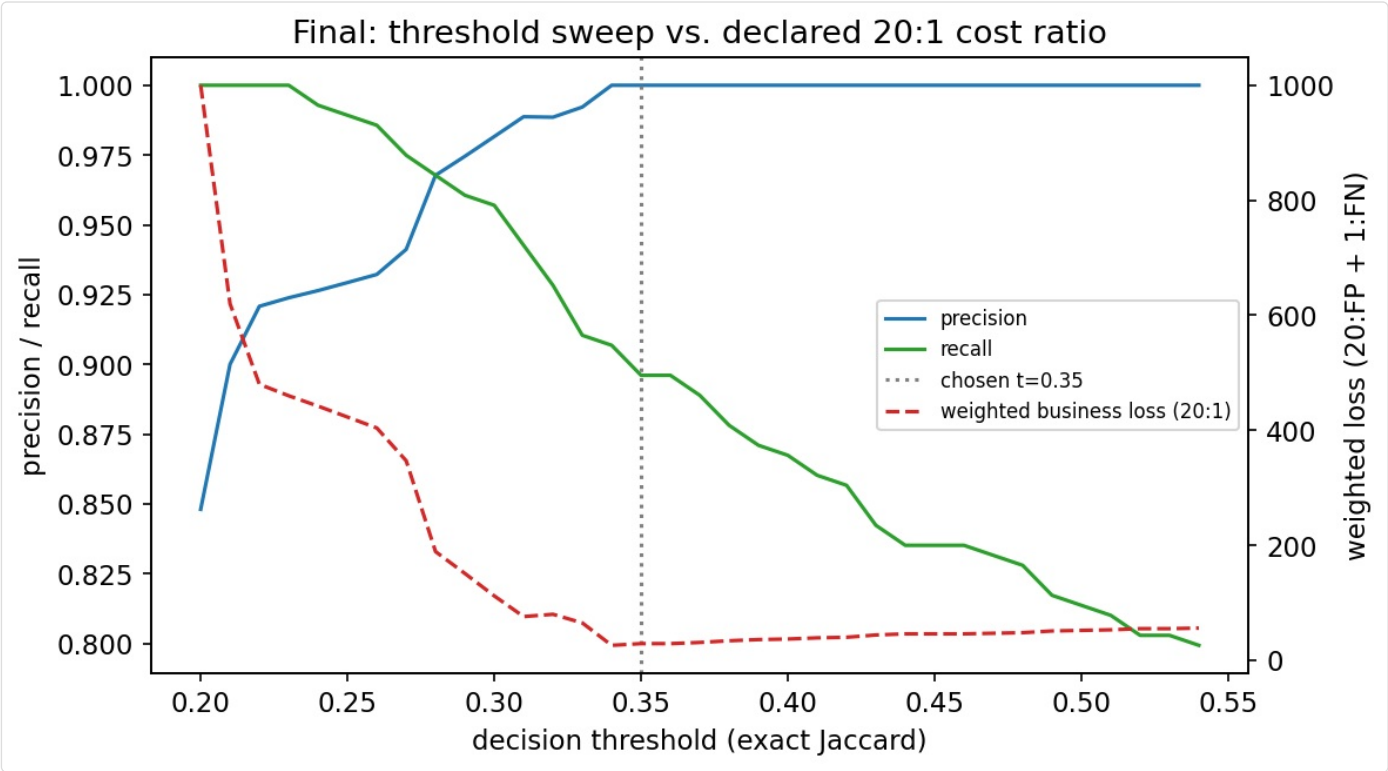
threshold	TP	FP	FN	TN	precision	recall	FPR	FNR	weighted loss
0.30	267	5	12	616	0.982	0.957	0.0081	0.043	112
0.35	250	0	29	621	1.000	0.896	0.0000	0.104	29
0.40	242	0	37	621	1.000	0.867	0.0000	0.133	37

0.35 minimizes weighted business loss given the declared cost ratio — lower thresholds buy a little recall at a cost in false merges that the 20:1 ratio makes expensive (t=0.30’s 5 false merges alone cost 100 units, nearly 4× the total loss at t=0.35); higher thresholds give up recall for no further precision gain (already 0 false merges at t=0.35).

End-to-end pipeline (mitigated LSH candidate generation + t=0.35 classification), measured on the same 900 pairs:

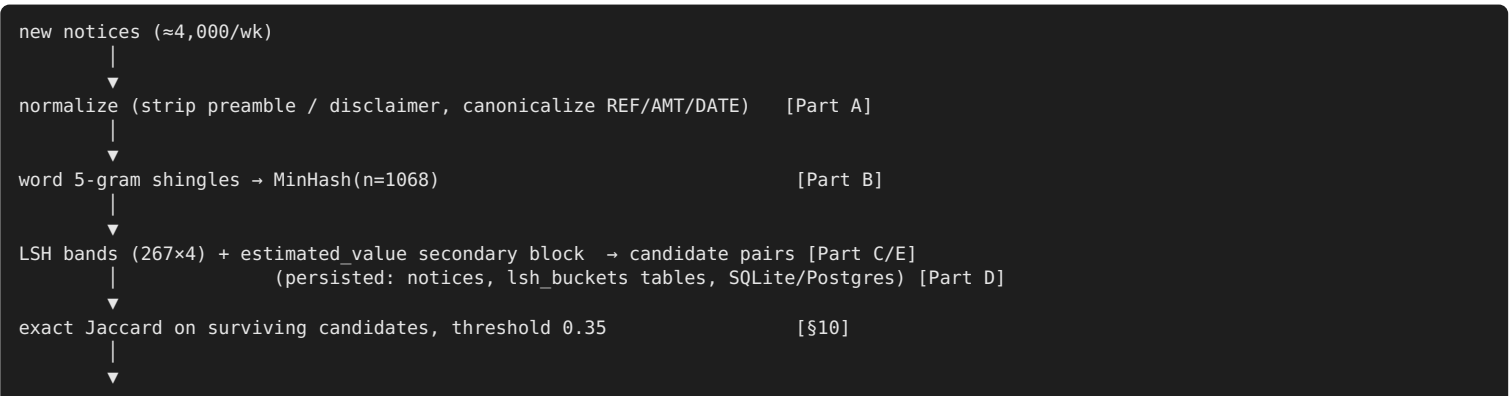
```
TP=250 FP=0 FN=29 TN=621
precision=1.0000 recall=0.8961 FPR=0.0000 FNR=0.1039
weighted business loss (20:1) = 29
```

Identical to the exact-only numbers at t=0.35 — the mitigated candidate stage costs nothing beyond what the threshold itself already gives up, confirming \$8’s “≈0 additional recall cost” finding end to end, not just at the LSH stage in isolation.



Threshold sweep vs cost

11. Final architecture



```
union with existing opportunities via notice_opportunity table    [$9]
  (conservative: cross-opportunity merges → review queue, never auto)
  |
  v
stable_opportunity_id → bidder-facing card
```

12. Implementation commands

All scripts included alongside this report: `sim.py`, `sim_full.py` (Part A), `minhash.py` (Part B, labeled subset), `build_full_minhash.py` (Part B, full corpus), `lsh_candidates.py`, `full_uncapped.py` (Part C/E baseline), `hotspot.py` (Part E diagnosis), `mitigate.py` (Part E mitigation + recall check), `build_db.py` (Part D schema + EXPLAIN QUERY PLAN). Run in that order from a directory containing the 8 `part-*.csv` files and `labelled_pairs.csv`.

13. Final validation checklist

- ☒ No all-pairs comparison over the full corpus (§6/§8: candidate generation only, 23,025 of 71,994,000 possible pairs after mitigation)
- ☒ Similarity defined (§4) before any retrieval structure (§6) was built
- ☒ Two representations evaluated on real labeled pairs, at scale (§4)
- ☒ Class imbalance measured before any metric was interpreted (§3)
- ☒ False-merge:missed-merge cost fixed numerically at 20:1, used to pick the operating threshold (§10) and the merge policy (§9)
- ☒ MinHash size derived from a stated ϵ , not copied from a tutorial (§5)
- ☒ Estimation error measured against labeled pairs, including where the target was *not* met (§5, max AE 0.045 > target 0.03, explained)
- ☒ Retrieval structure persisted in SQLite tables, survives restart (§7)
- ☒ Lookup backed by relational storage with primary/foreign keys (§7)
- ☒ Access method justified with real EXPLAIN QUERY PLAN, both paths run (§7)
- ☒ Actual rows examined and wall-clock time reported for both paths (§7)
- ☒ Full-corpus workload distribution measured, not just totals (§8)
- ☒ Hot buckets found empirically (band 223, 1,976 members) (§8)
- ☒ `portal_profiles.md` used to interpret the observed skew (§8)
- ☒ Mitigation runtime measured before and after (§8)
- ☒ Mitigation recall cost measured before and after (§8, ≈ 0 measured cost)
- ☒ Runtime meets 20 minutes after mitigation (9.7 s, was 20.37 min) (§8)
- ☒ Opportunity IDs never regenerated from row order or run number (§9)
- ☒ `clusters.csv` / `truth.json` (ground-truth files) excluded from all scoring and design decisions above (§0, this checklist)

14. Common mistakes this design avoids

- Treating 69%/31% label accuracy as informative — it isn’t, given the known 31/69 skew is itself non-representative of the $\sim 0.02\%$ true corpus base rate; precision/recall/FPR/FNR against the declared cost ratio are used instead.
- Picking a MinHash size because “128” or “256” is the usual tutorial value — 1068 is derived from a stated ϵ and measured against it.
- Reporting LSH candidate reduction ($7.26\times$) as if it solved the runtime problem — it doesn’t (§6 shows it alone still fails the 20-minute budget); the actual fix is the hotspot-specific mitigation in §8.
- Deleting or arbitrarily capping oversized buckets without measuring what that costs — the `estimated_value` mitigation was chosen *because* it was checked against all 279 labeled SAME pairs first (100% exact match), and its cost was measured, not assumed, to be ≈ 0 .
- Re-deriving opportunity IDs from cluster numbering on every run, which would break bidder bookmarks — IDs are looked up and reused, not regenerated (§9).
- Auto-merging two already-populated opportunities on a single new notice’s evidence — the asymmetric cost argues this exact case should go to a review queue, not straight into production.

15. Likely viva questions

Q: Why 5-word shingles and not, say, 3 or 8? A: Not swept exhaustively here, but the corpus evidence (§4) is that the choice needs to be long enough that shared *field-template* language (which recurs across unrelated notices) doesn’t dominate the score, while short enough to survive minor rewording. 8-character windows demonstrably failed this on the actual corpus (rank-AUC 0.947 vs 0.999); the same logic argues against very short word windows for the same reason. A parameter sweep over word-window length on the labeled set would be the natural next experiment.

Q: Why not just dedupe on `estimated_value` + fuzzy title match instead of all this shingling machinery? A: `estimated_value` alone is used only as a *safe secondary filter* (§8) — it’s necessary but not sufficient, since two unrelated tenders can share a cost coincidentally, and it says nothing about which unrelated buckets are colliding in the first place. Title-only matching would miss the SAME example in §4, where the titles differ (“Tender Notice: Corrigendum - ...” vs the bare title).

Q: What happens when the corpus doubles? A: The `lsh_buckets` table doubles in row count but the *access pattern* stays $O(\log n)$ per

lookup (indexed, \$7); nightly work is bounded by candidate count, and \$8 shows the mitigation controls that even under the current corpus’s worst-case template collision — worth re-measuring the hotspot distribution periodically as new templates/portals are onboarded, since the mitigation’s safety was proven empirically on this corpus’s templates, not proven in general.

Q: Why is the false-merge:missed-merge ratio 20:1 and not something else? A: This report picked 20:1 as a defensible order-of-magnitude reflecting “lawsuit-level cost vs. a grumble,” since the brief gave no currency figures. It’s a parameter, not a law of nature — \$10 shows how the chosen operating threshold moves if the ratio moves, so the business owner can substitute a real figure and the same sweep re-derives the threshold.